

Activity 03: Modeling & Forecasting

SISMID Lab

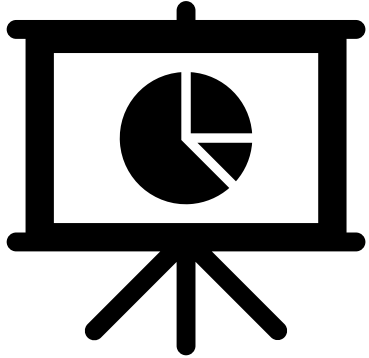
July 22 through July 24, 2026

A Human-AI Teaming Approach to Infectious Disease Modeling

Lecture Overview

Topic	Slides
1. General Housekeeping	3
2. Lab Timeline	4
3. Task Overviews	5
4. Goal Output	6
5. Joint Instructor and Class Exercise	7
6. Task 0: Set-Up	8–11
7. Task 1: Training and Forecast Period	12–16
8. Task 2: Baseline Model Specification	17–22
9. Task 3: Forecasting	23–42
10. Task 4: Visualization	43–47
11. Task 5: Extensions	48–61

General Housekeeping



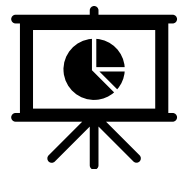
Any slide with this icon contains the **overview** and **expected outputs** for the included tasks. All outputs listed on these slides should be uploaded to the course GitHub repository for review.



Any slide with this icon indicates that it is **time to complete a hands-on activity**. Carefully follow the instructions provided, complete the required task, and verify your results.

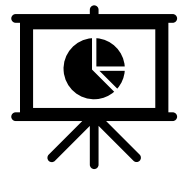
Lab Timeline





Task Overviews

Task	Description
Task 0: Set-Up	Set up the environment and load and validate the input data.
Task 1: Specifying the Training and Forecast Period	Define the training and testing periods and set up an expanding window so each forecast trains on all data available up to that week.
Task 2: Baseline Model Specification	Specify a baseline ARIMA (Auto Regressive Integrated Moving Average) model and write the model specification rules for the AI to follow.
Task 3: Forecasting	Generate forecasts from the reference date and target end dates, extract the median and 95% prediction interval bounds, and prepare the output in FluSight format.
Task 4: Visualization	Visualize the forecast results and write the visualization rules that define how the plots should look.
Task 5: Extensions	Expand to the full FluSight format by adding the 2- and 3-week-ahead forecasts and iterating on the prompt.



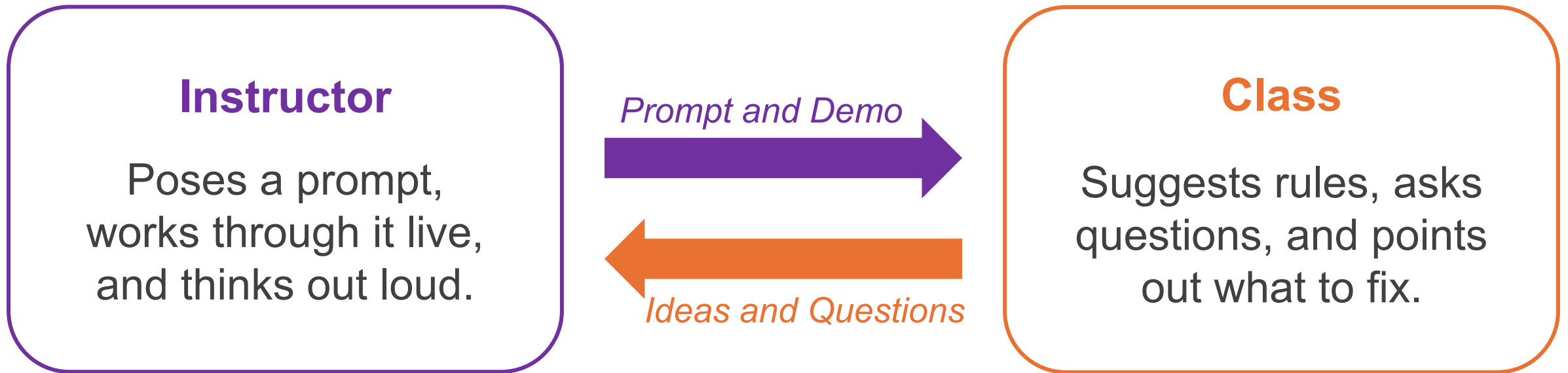
Goal Output

From this activity you should produce two outputs: a **FluSight-format forecast file** and a **forecast figure**. An AI agent generates them from the cleaned weekly admissions data, guided by the rules you write in rules.md.

Activity Output	Description
flusight_forecasts.csv	A FluSight-format forecast file with 1-, 2-, and 3-week-ahead quantile forecasts (median and prediction interval bounds) for U.S. influenza admissions.
forecast_vs_observed.png	A plot of the forecast showing the median and 95% prediction interval against the observed weekly admissions data.

A Joint Instructor and Class Exercise

We tackle the prompts together, live. Then, we will trade ideas back and forth, refining the rules until the AI produces what we want.



We keep this back and forth going until the output looks right, then you try it on your own.

Task 0: Set-Up



Forecasting

- We first need to put rule(s) in place to **create** and **save** the R script. As this is an important piece to set-up the rest of what we will do for this activity, this code will sit at the **very top** of our rules. **Add this rule** under the # Forecasting header:

Save all code in this section to the following output script:

```
**Output folder:** `output/scripts/03_forecast.R`
```

- The output folder **may not yet exist** in your working directory; we can put a rule in place to guard against this:

If the folder pathway is not present, create the folder pathway prior to saving the script.



1. Input Data

- Next, we need to provide details related to what data we want to work with throughout the script (i.e., data we created in the previous activities).
- To do so, all we need to do is give the AI the **file path** to the data. This ensures that the exact file we want is used as the remainder of the script is created.
- First, **add** the 1. Input Data header under # Forecasting. Then, **copy** over the following rule:

```
Read in the input data set from  
`output/data/01_cleaning/cleaned_flu_admissions.csv`.
```



Input Data Validations

- AI is **not perfect** by any means, so we want to build in some validations and error messages to help us catch any mistakes that might occur (e.g., file structure, parsing issues, etc.).
- Under the input rule, **include** the following validations rules:

Ensure the column names and column types/formats match the following:

- ``week``: Date column, YYYY-MM-DD
- ``location``: Character; all values should be "US"
- ``value``: Numeric

If any columns have violations please first try to parse to the correct format, and then notify the user. If any columns can not be parsed, please return the following error:

`'The {column} could not be parsed.'`

Task 1: Specifying the Training and Forecast Period

The Plan: Training & Testing Periods

TRAINING PERIOD

Seasons before 2025-2026

Fit the model on **all observed weeks** strictly before the first week of the testing period present in the data. Use these rows as the fixed initial training set; subsequent forecasts will use an **'expanding window'**.



TESTING PERIOD (Rolling)

2025-2026 Season

Use the defined **2025-2026 season** (See Previous Slides) as the testing period.

Forecasting Horizon: 1-Week Ahead (`horizon = 1`)

Expanding Window

Forecasts are produced with an **expanding window**. For each target week in the testing period, the model is refit on all observed weeks that fall strictly before that target week. The initial training data is the set of weeks strictly before the first testing-week present in the data; as each test week becomes observed it is added to the fitting window for subsequent targets.

Adding in Diagnostics (a.k.a. Checks)

As we build this out, it's important that we include **code checks** along the way to ensure that things we specified in the rules.md are being interpreted by the AI as we intended.

Some Examples:

****Validations:**** Print the `Training Period` start date and rolling window end dates.

****Validations:**** Print the `Testing Period` start date, and each forecast date.

****Validations:**** Print the specified forecasting horizon that will be used.



Now it's your turn!

We will work through the following steps together to add the next section to our rules.md document.

1

Create the `## 3. Training and Testing Periods` section header in rules.md.

2

Write detailed rules to specify the training and testing period information; including the '**expanding window**' set up (we will do this together).

3

Regenerate **agents.md** from your updated rules document

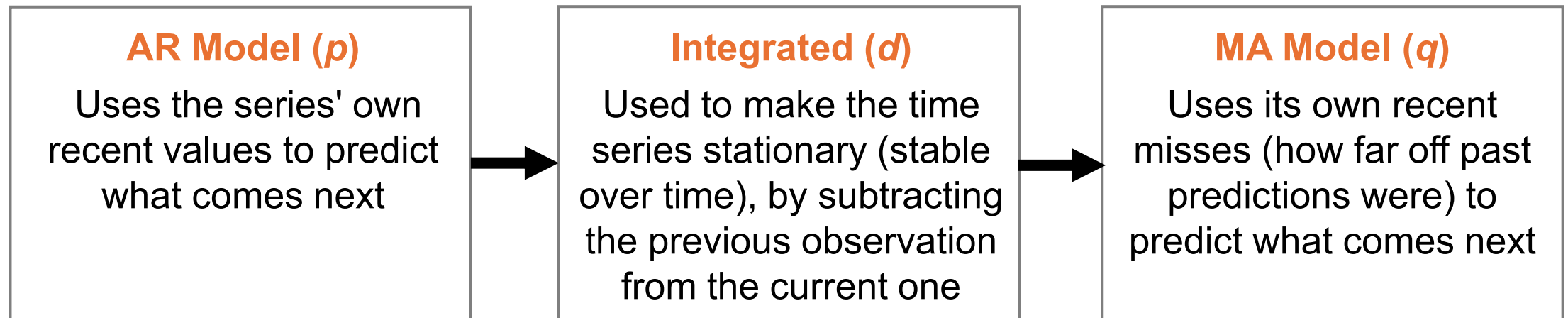
4

Use the **agents.md** file and Copilot (or your preferred AI) to **generate output/scripts/03_forecast.R**. Run the script and check the output.

Task 2: Baseline Model Specification

Auto Regressive Integrated Moving Average (ARIMA)

A group of models that uses a time series' **past values** and **past forecast errors** to explain the current values of the series, with the goal of producing forecasts.



Assumes: The series is stationary after differencing (stable mean and variance over time).

Model Specification Rules

Rules for Model Specification

	Rule	Description
Package	<code>forecast;</code> <code>forecast::<u>auto.arma()</u></code>	Use the <u><code>auto.arma()</code></u> function from the <code>forecast</code> package.
Response Variable	The influenza admissions series only . Sort the training window by <code>`week`</code> , extract the ordered value vector, and use that as the only input. No covariates or external regressors are permitted.	Use only the influenza admissions values. Put the training data in date order by <code>`week`</code> , pull out that ordered list of values, and feed only that to the model. Nothing else is added (i.e., no other variables or outside predictors).
Model Details	A non-seasonal ARIMA whose order is selected automatically by <u><code>auto.arma()</code></u> . The series is passed as a plain numeric vector, so <u><code>auto.arma()</code></u> performs its own <code>`(p, d, q)`</code> selection.	The model is a non-seasonal ARIMA, a standard forecasting method that predicts future values from the recent history of the series, without fitting a repeating yearly pattern. We hand <u><code>auto.arma()</code></u> just the plain list of counts and let it automatically choose the settings that fit the data best.

Adding in Diagnostics (a.k.a. Checks)

As we build this out, it's important that we include **code checks** along the way to ensure that things we specified in the rules.md are being interpreted by the AI as we intended.

Some Examples:

****Validations:**** Print confirmation that the training window is sorted ascending by `week` with no missing or duplicate weeks.

****Validations:**** Print confirmation that the `week` index is evenly spaced (no skipped weeks); halt if this fails.

****Validations:**** Print the `(p, d, q)` order that `auto.arima()` selected at each refit step.



Now it's your turn!

We will work through the following steps together to add the next section to our **rules.md** document.

- 1 **Create** the `## 4. Model Specification` **section header** in `rules.md`.
- 2 **Write detailed rules** to fit the model using the expanding window approach and put the checks into place (we will do this together).
- 3 Regenerate **agents.md** from your updated rules document
- 4 Use the **agents.md** file and Copilot (or your preferred AI) to generate **output/scripts/03_forecast.R**. Run the script and check the output.

Task 3: Forecasting

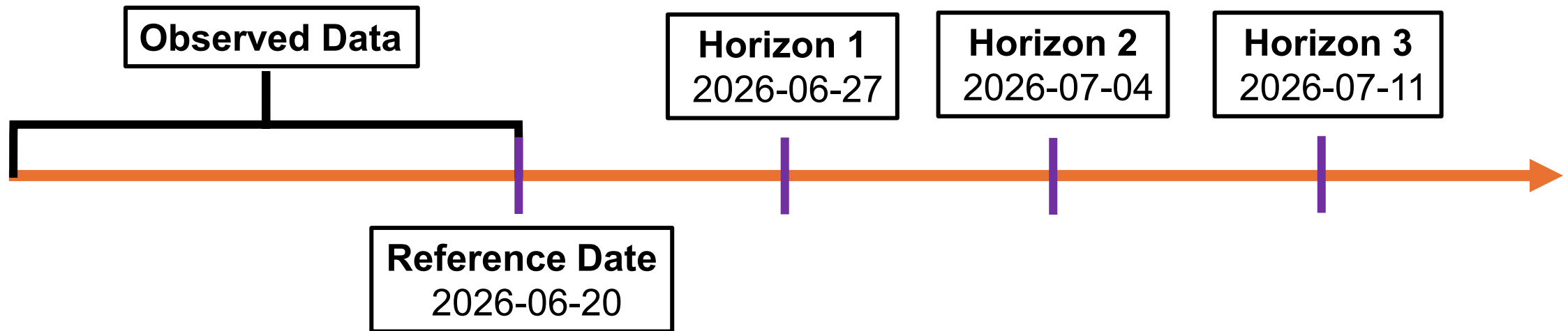
What does our data look like?

week	location	value
2020-08-08	US	0
2020-08-15	US	0
...		
2026-06-06	US	1011
2026-06-13	US	832
2026-06-20	US	703

What are our **reference dates** (i.e., cut off for input data) and what are our **target end dates** (i.e., forecast dates)?

Reference Dates

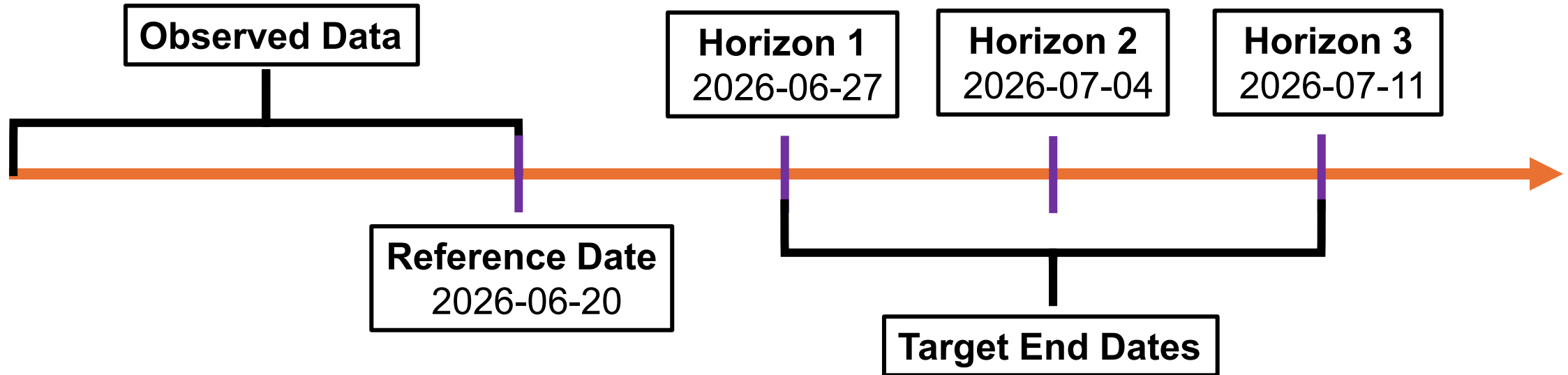
Reference dates are the **last week of observed data** we include in our training period (i.e., model input). A week is defined as Sunday to Saturday, with the Saturday used for the label.



For example, if our most recent data is the week of **2026-06-20**, that's our reference date, and we forecast the *next* week from it.

Target End Dates

Target end dates are the **weeks we are producing forecasts for**. A week is defined as Sunday to Saturday, with the Saturday used for the label.



For example, if our reference date is 2026-06-20, our target end dates are **2026-06-27**, **2026-07-04**, and **2026-07-11** (the 1-, 2-, and 3-week-ahead forecasts).

Things to Consider

- How do we calculate our reference date?
 - **Hint:** Compute the reference date as the calendar date exactly one week before the target date: `reference_date = as_date(t) - days(7)`.
- Where do we want to use the reference date specification?
 - **Hint:** Use this date (`reference_date`) for both training-window selection and for output.
- How do we want to explicitly format our dates in the output?
 - **Hint:** When writing CSVs, format `reference_date` and `target_end_date` as ISO dates (``YYYY-MM-DD``).

Be explicit about dates. R can handle dates unpredictably; by being explicit we can stop issues and unexpected behavior before they happen.

Adding in Diagnostics (a.k.a. Checks)

Some Examples:

****Validations:**** Print confirmation that `reference_date` and the target `t` are both Date objects (not character or POSIXct); halt if either fails the class check.

****Validations:**** Print confirmation that `target_end_date - reference_date` equals exactly 7 days for every row; halt if this fails.

****Validations:**** Print confirmation that the maximum week in the training window is `<= reference_date` and strictly less than the `target_end_date` (no look-ahead leakage); halt if this fails.



Now it's your turn!

Together, we will update the rules to include the date formatting requirements discussed on the previous slide (contains **hints** related to formatting).

- 1 **Create** the `## 5. Forecast Generation` **section header** in `rules.md` and add the statement `For each target week `t` in the testing period that is present in the data:` **under it**.
- 2 **Write detailed rules** to specify how to handle dates in list item one (i.e., under the statement from Step 1; add `1.` and your rules). As part of this step, **add in validation checks**.
- 3 Regenerate **`agents.md`** from your updated rules document.
- 4 Use the **`agents.md`** file and Copilot (or your preferred AI) to re-generate **`output/scripts/03_forecast.R`**. Run the script and check the output.



We have our date rules, now what?

- **Data leakage** happens when a model is tested on data used in training (i.e., it "predicts" values it was handed, so the forecast is better than it would be in practice when you don't actually see the data).
- To prevent this, we're explicit about the cutoff: only data on or before the **reference date** goes in as input, so the model never sees the weeks it's forecasting.
- Add the following under the date rule(s):

```
2. Fit the ARIMA model (Rule 4) on all rows with  
  `week <= reference_date` (expanding-window).
```

Why include the data leakage rule again? AI assistants can be forgetful. When we ask the AI to generate a new section of code, it does not always carry forward the constraints we set earlier, so an important rule can quietly get dropped. The cutoff, using only data on or before the reference date, is what keeps the forecast honest, so we restate it right where the forecasts are generated. That keeps it front and center for the AI and guards against the model training on the very weeks it is supposed to predict, which would make the forecast look better than it really would be in practice.



Forecasting (1-Week Ahead)

output_type	output_type_id	value	Description
quantile	0.025	#	Lower Bound (95% PI)
quantile	0.5	#	Median
quantile	0.975	#	Upper Bound (95% PI)

How do we get the values?

```
3. Forecast with `forecast(fit, h = 1, level = 95)`.
```


Extracting the Median & 95% PI Bounds

```
forecast(fit, h = 1, level = 95)
```

Lower Bound
(fcst\$lower)

Point Forecast
(fcst\$mean)

Upper Bound
(fcst\$upper)

Things to think about when writing rules.

What pieces of information do we need to extract from the forecast call? What format will the data come back in (**Hint:** The lower and upper bounds come back with one column per level)? Are there any restrictions we want to place on the results (**Hint:** Rounding, Truncating at 0)?

Adding in Diagnostics (a.k.a. Checks)

Some Examples:

****Validations:**** Confirm mean, lower, and upper are all finite (no NA/NaN/Inf) at each forecast step; halt if not.

****Validations:**** Confirm the values for each quantile are non-decreasing (ex. value for $0.025 \leq 0.5 \leq 0.975$) within every target week. Halt on any violation.

****Validations:**** Confirm all value entries are non-negative integers.



Now it's your turn!

Together, we will add a rule for extracting the median and 95% PI Bounds. Pay particular attention to the **hints** provided on the previous slide.

- 1 Using the hints from the previous slide, **write the rule** (not the code) for extracting the median and 95% PI bounds from the forecast function (Rule 4). Make sure to include a statement about the output structure, any rules about results, and code checks (**Hint:** See previous slides).
- 2 Regenerate **agents.md** and Use the **agents.md** file and Copilot (or your preferred AI) to regenerate **output/scripts/03_forecast.R**. Run the script and check the output.

Note: In a few places we hand the AI an **exact function** or code snippet. This does not defeat the purpose of AI-assisted coding. When I already know a function that works well for the task, naming it points the AI to the right approach instead of leaving it to guess, and the AI still writes and connects the surrounding code. Bringing that kind of domain knowledge is the human side of the human-AI team.

Preparing File in FluSight Format

What is the FluSight format?

Column	Description	Entry Type	Acceptable Values
reference_date	The last day of the week the forecast(s) are submitted to the challenge. It is the Saturday following the forecast due date.	ISO Format Date	YYYY-MM-DD
target	The target of the forecast.	Character	wk inc flu hosp
horizon	The number of weeks between the reference_date and the target_end_date.	Numeric	[1]
target_end_date	The last date of the retrospective/nowcast/forecast target's week (Saturday).	ISO Format Date	YYYY-MM-DD
location	FIPS codes for U.S. states as well as "US" for national forecasts the corresponding row is associated with.	Character	US
output_type	The type of value shown.	Character	quantile
output_type_id	The quantile label for the value in the corresponding row.	Numeric	0.01, 0.025, 0.05, 0.1, 0.15, 0.2, 0.25, 0.3, 0.35, 0.4, 0.45, 0.5, 0.55, 0.6, 0.65, 0.7, 0.75, 0.8, 0.85, 0.9, 0.95, 0.975, and 0.99
value	The non-negative numbers indicating the quantile prediction for the corresponding row.	Numeric	Integer

First, Handling the Quantiles (Wide-to-Long)

Our Current Format

0.025	0.5	0.975
VALUE	VALUE	VALUE



Our Goal Format

<code>output_type_id</code>	<code>value</code>
0.025	#
0.5	#
0.975	#



How do we get there?

Add the following (Including the table):

4. ****Reshape wide → long, one row per quantile.**** Map each forecast piece to an ``output_type_id``: ``mean`` → ``0.5`` (median); each level's lower/upper bound → its two quantiles, per the table. Only include the levels below.

Coverage Level	LB Quantile (<code>`output_type_id`</code>)	UB Quantile (<code>`output_type_id`</code>)
95	0.025	0.975

Why do we need the table?

`forecast()` and FluSight describe the same thing in two different languages. When you ask for a "95% interval," `forecast()` gives you a **lower** and an **upper edge** — those edges are the 0.025 and 0.975 quantiles that FluSight asks for. Think of the table as your & the AI's **translation guide** between the two.

Hint (for later): Want more quantiles? Update **two things together** — add a row here (table) **and** set `level` in the forecast call to match. After that, everything downstream follows automatically.

Column	Description	Entry Type	Acceptable Values
reference_date	The last day of the week the forecast(s) are submitted to the challenge. It is the Saturday following the forecast due date.	ISO Format Date	YYYY-MM-DD
target	The target of the forecast.	Character	wk inc flu hosp
horizon	The number of weeks between the reference_date and the target_end_date.	Numeric	[1]
target_end_date	The reference date for the forecast.	Character	YYYY-MM-DD
location	The location of the forecast. The corresponding row is associated with.		US
output_type	The type of value shown.	Character	quantile
output_type_id	The quantile label for the value in the corresponding row.	Numeric	0.01, 0.025, 0.05, 0.1, 0.15, 0.2, 0.25, 0.3, 0.35, 0.4, 0.45, 0.5, 0.55, 0.6, 0.65, 0.7, 0.75, 0.8, 0.85, 0.9, 0.95, 0.975, and 0.99
value	The non-negative numbers indicating the quantile prediction for the corresponding row.	Numeric	Integer



Now it's your turn!

On your own, add the **rules for formatting** the forecast in FluSight format and **saving the file** (Hint: look at the column names and descriptions provided on the previous slide).

1

Using a table format (see **Reshape wide** slide), **create a table** with the Column names and Type/Value for each field required for FluSight. Include this table as part of **Rule 5**. Use the table on the previous slide to guide you.

2

Save the file to `output/data/03_forecast/flusight_forecasts.csv`; create the folder if it is absent.

3

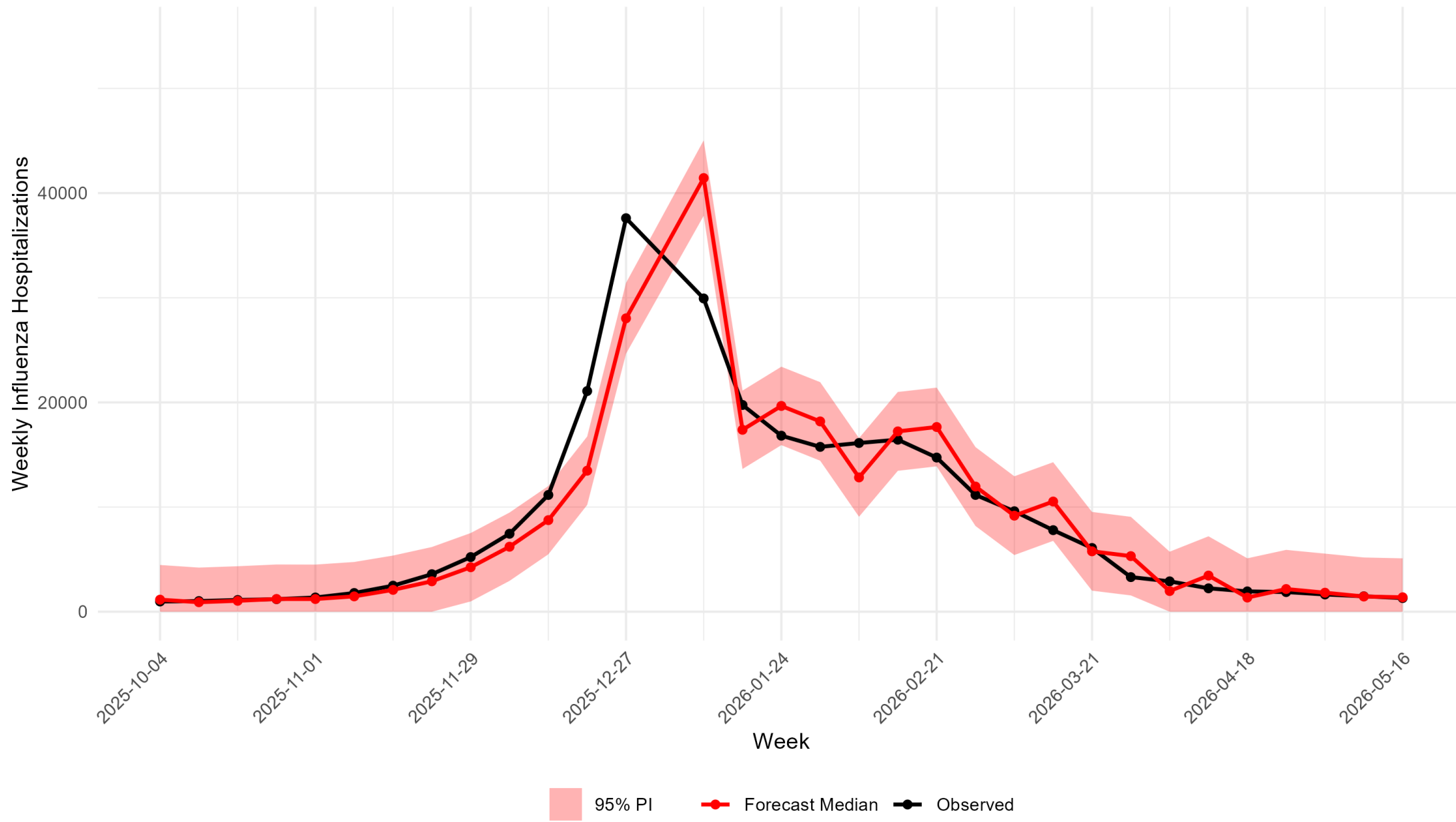
Regenerate **agents.md** from your updated rules document and use the agents.md file and Copilot (or your preferred AI) to re-generate **output/scripts/03_forecast.R**. Run the script and check the output.

Task 4: Visualization

Visualizing our Results

- As a final step, we will **visualize our results**. A quick plot lets us catch any immediate issues (misaligned dates, wildly wide intervals, forecasts that ignore the trend) and get a feel for overall performance.
- Remember, the rules for figures should **be specific** enough that someone (or an AI) could produce the plot without asking any follow-up questions.
- We'll work **together** to build our final figure.

USA 1-Week-Ahead Influenza Hospitalization Forecast (2025-26 Season)



Writing the Visualization Rules

Things to think about:

Element	Description
Plot type & Plot Elements	line chart, bar chart, scatter, etc.
X-Axis and Y-Axis	variable, label text, date format, start at zero?
Title	exact title text for the figure
Colors and highlights	line color, shading, annotation colors
Annotations and markers	peak markers, reference lines, text labels
Legend	Legend elements, labels, colors
Output file name	output/figures/03_forecast/forecast_vs_observed.png
Save Requirements	DPI



Now it's your turn!

We will work through the following steps together to add that next section to our **rules.md** document.

1

Create the `## 6. Forecast Figure` **section header** under `## 5. Forecast Generation section` header in `rules.md`.

2

Write detailed rules to obtain the forecast visual. Make sure to specify plot types, axes, colors, annotations, file names, etc. (vague rules = vague output).

3

Regenerate **agents.md** from your updated rules document

4

Use the **agents.md** file and Copilot (or your preferred AI) to update **output/scripts/03_forecast.R**. Run the script and **check the output**.

Task 5: Extensions

Expanding to full FluSight Format

- Our current forecasts **do not** capture all horizons and quantiles required by FluSight.
- The remainder of this activity will focus on expanding our current framework to all **23 quantiles** and **forecasting horizons** required by the challenge.

	Current	Goal
Horizons	1-Week	1 Week - 3 Weeks
Quantiles	0.025, 0.5, 0.975	0.01, 0.025, 0.05, 0.1, 0.15, 0.2, 0.25, 0.3, 0.35, 0.4, 0.45, 0.5, 0.55, 0.6, 0.65, 0.7, 0.75, 0.8, 0.85, 0.9, 0.95, 0.975, and 0.99

Adding the 2- and 3-Week Forecasts

- We reference our horizon (1-week) in many places — across the rules document and throughout the code. Rather than manually updating each one, **we can use AI to help**.
- There are two ways we can go about it: (1) Manually update the rules document or (2) Work with AI to update the rules document and subsequent code.
- For this exercise, we will be **using AI to help** us update the rules to get exactly what we want.

Prompting & Things to Consider

- We are a **human-AI team**, so we need to make sure we give the AI the domain instruction it needs to help us execute our updates.
- First, let's think about the **different pieces** of our forecasting system that incorporates our forecasting horizon specification:

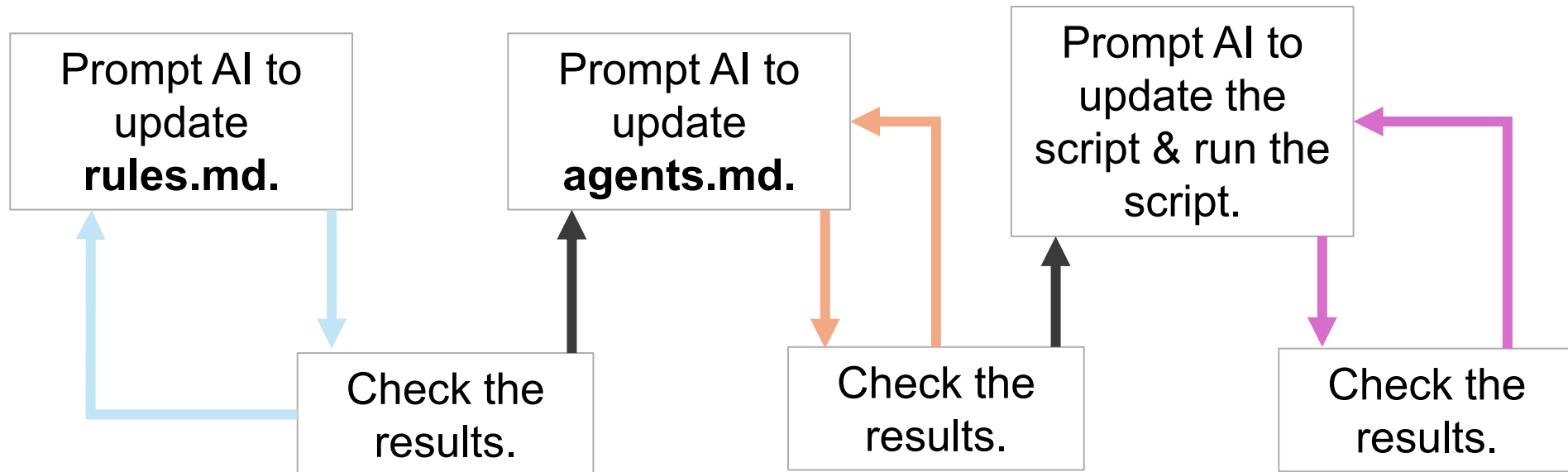
Forecasting

Visualization

Data Formatting

Updating is Often Iterative

- AI is **NOT** perfect and may return different results each time you run it. It often may take a bit of **back-and-forth** to get to the final product you desire.
- For each iteration, make sure to check **all** changes the AI makes.



Adding in Diagnostics (a.k.a. Checks)

AI can make mistakes, so we need to make sure to stay in the loop!

****Validations:**** Confirm intervals get wider (or stay equal) as the horizon grows: width at $h=3 \geq h=2 \geq h=1$. Print "[val] intervals widen with horizon: OK"; stop() otherwise.

****Validations:**** Confirm $\text{target_end_date} == \text{reference_date} + 7 * \text{horizon}$ for every row. Print "[val] target dates correct: OK"; stop() otherwise.

****Validations:**** Confirm each reference date is fit once, with all three horizons taken from that single forecast. Print "[val] one fit, three horizons: OK"; stop() otherwise.



Writing the Prompt

- We are going to work with AI to help us write the **updated rules**.
- We want to make sure we are **explicit** about the changes we want and review **ALL** changes it suggests.
- Remember, we need to ask for changes **in all places** where we specify the forecast horizons:

First Part of Prompt: Introducing the Ask

I want to extend my forecasting to include 1-, 2-, and 3-week forecasting horizons. Please update the rules document where we forecast, prepare the data file, and visualizations.



Writing the Prompt

Second Part of Prompt: Data Format

Please keep the flusight format for the data file.

Third Part of Prompt: Visualizations

For the visualizations, I want each horizon to have their own line like what we do currently for 1-week horizons. Make sure the colors are distinct, and the legend clearly includes all graph elements.



Writing the Prompt

Final Part of Prompt: Validations

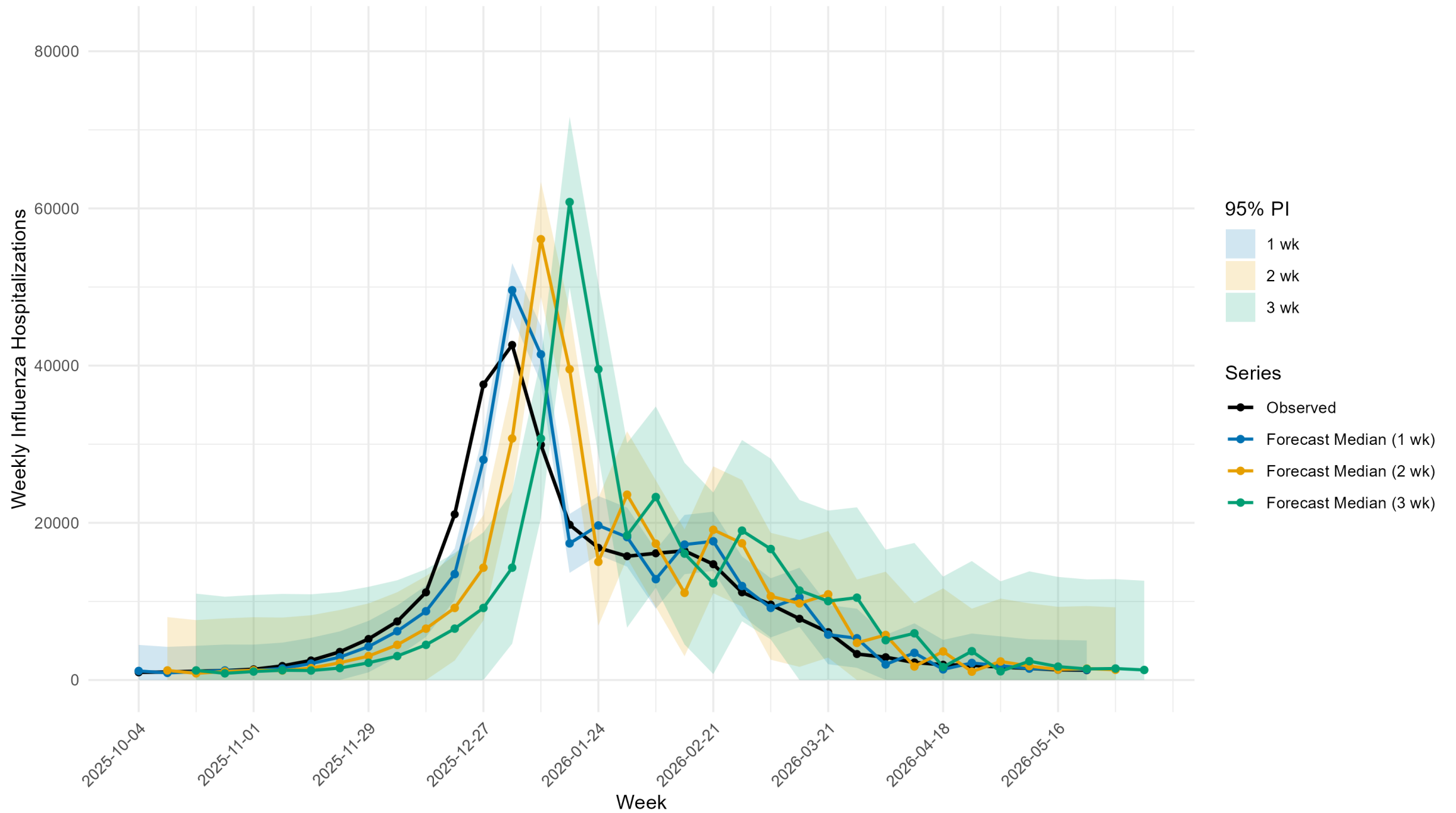
Please also include the following data checks:

Validations: Confirm intervals get wider (or stay equal) as the horizon grows: width at $h=3 \geq h=2 \geq h=1$. Print "[val] intervals widen with horizon: OK"; stop() otherwise.

Validations: Confirm `target_end_date == reference_date + 7 * horizon` for every row. Print "[val] target dates correct: OK"; stop() otherwise.

Validations: Confirm each reference date is fit once, with all three horizons taken from that single forecast. Print "[val] one fit, three horizons: OK"; stop() otherwise.

USA 1-, 2-, & 3-Week-Ahead Influenza Hospitalization Forecast (2025-26 Season)



Updating the Quantiles

- Now that we have updated our forecasting horizons, we now need to update our pipeline to capture all **23 quantiles**

	Current	Goal
Quantiles	0.025, 0.5, 0.975	0.01, 0.025, 0.05, 0.1, 0.15, 0.2, 0.25, 0.3, 0.35, 0.4, 0.45, 0.5, 0.55, 0.6, 0.65, 0.7, 0.75, 0.8, 0.85, 0.9, 0.95, 0.975, and 0.99

- For this exercise, we will be **using AI to help** us update the rules to get exactly what we want.



Writing the Prompt

- We are going to work with AI to help us write the **updated rules** around adding additional quantiles.
- We want to make sure we are **explicit** about the changes we want and review **ALL** changes it suggests.

Prompt: Updating the Quantiles

Now I want to update the quantiles we are forecasting. Please expand our forecasts and data files to capture the following quantiles (see attached). Please update the rules document to start.

Then, update the agents.md and 03_forecast.R.



Writing the Prompt

Final Part of Prompt: Validations

Please also include the following data checks:

Validations: For each reference date and horizon, confirm the quantiles are non-decreasing across every level: $0.01 \leq 0.025 \leq \dots \leq 0.975 \leq 0.99$. Print "[val] quantiles non-decreasing: OK"; stop() otherwise.

Validations: Confirm every reference-date/horizon has the exact required set of quantile levels – none missing, none extra. Print "[val] all quantile levels present: OK"; stop() otherwise.

Validations: Confirm the median (0.5) equals the point forecast and symmetric pairs are equidistant from it (e.g. 0.025 and 0.975). Print "[val] median centered, quantiles symmetric: OK"; stop() otherwise.

```
output > data > 03_forecast > flusight_forecasts.csv > data
```

	reference_date	target	horizon	target_end_date	location	output_type	output_type_id	value
1	2025-09-27	wk inc flu hosp	1	2025-10-04	US	quantile	0.01	0
2	2025-09-27	wk inc flu hosp	1	2025-10-04	US	quantile	0.025	0
3	2025-09-27	wk inc flu hosp	1	2025-10-04	US	quantile	0.05	0
4	2025-09-27	wk inc flu hosp	1	2025-10-04	US	quantile	0.1	0
5	2025-09-27	wk inc flu hosp	1	2025-10-04	US	quantile	0.15	0
6	2025-09-27	wk inc flu hosp	1	2025-10-04	US	quantile	0.2	0
7	2025-09-27	wk inc flu hosp	1	2025-10-04	US	quantile	0.25	9
8	2025-09-27	wk inc flu hosp	1	2025-10-04	US	quantile	0.3	263
9	2025-09-27	wk inc flu hosp	1	2025-10-04	US	quantile	0.35	498
10	2025-09-27	wk inc flu hosp	1	2025-10-04	US	quantile	0.4	721
11	2025-09-27	wk inc flu hosp	1	2025-10-04	US	quantile	0.45	937
12	2025-09-27	wk inc flu hosp	1	2025-10-04	US	quantile	0.5	1150
13	2025-09-27	wk inc flu hosp	1	2025-10-04	US	quantile	0.55	1362
14	2025-09-27	wk inc flu hosp	1	2025-10-04	US	quantile	0.6	1578
15	2025-09-27	wk inc flu hosp	1	2025-10-04	US	quantile	0.65	1801
16	2025-09-27	wk inc flu hosp	1	2025-10-04	US	quantile	0.7	2036
17	2025-09-27	wk inc flu hosp	1	2025-10-04	US	quantile	0.75	2290
18	2025-09-27	wk inc flu hosp	1	2025-10-04	US	quantile	0.8	2573
19	2025-09-27	wk inc flu hosp	1	2025-10-04	US	quantile	0.85	2902
20	2025-09-27	wk inc flu hosp	1	2025-10-04	US	quantile	0.9	3317
21	2025-09-27	wk inc flu hosp	1	2025-10-04	US	quantile	0.9	3317